Assignment – 2
CSCI 6883.81 / CSCI 3461.51 Data Mining and Applications
Total Points: 100

Purpose: This assignment provides a deeper understanding and practical experience in pattern mining and association concepts.

Requirements & Submission:

1. The answers to this assignment can be submitted electronically on Blackboard.
2. Don't forget to write *the question number that corresponds* to your answer.
3. The file names must be your first name, followed by Assignment 2. For example, if your first name is "**avi**", rename your files as "**avi.Assignment-2.pdf**".
4. You must submit the Python code in the "**avi.Assignment-2.py**" file.
5. Must read weeks 5, 6, 7, and 9 of the contents of the syllabus from a textbook or reference book or online, as well as **class notes**. Some answers require more details than just what we had discussed in lecture slides or available in the book.

Question (1) (Total: 15+20 = 35 points)
The following questions are related to the data cleaning, package applications (Pandas, NumPy, MatPlotLib, etc.), data frame, array, list, dictionary, factor, import, and export of *.csv or *.txt files. Compute the following in Python.

i) (**5 points**) Create integer numbers between 25 and 150.

Generate + Code + Markdow

```python
#imoporting packages
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import pickle
```

[1] ✓ 1.3s

```python
numbers1 = np.arange(25, 151)
print(numbers1)
```

[2] ✓ 0.0s

```
[ 25  26  27  28  29  30  31  32  33  34  35  36  37  38  39  40  41  42
  43  44  45  46  47  48  49  50  51  52  53  54  55  56  57  58  59  60
  61  62  63  64  65  66  67  68  69  70  71  72  73  74  75  76  77  78
  79  80  81  82  83  84  85  86  87  88  89  90  91  92  93  94  95  96
  97  98  99 100 101 102 103 104 105 106 107 108 109 110 111 112 113 114
 115 116 117 118 119 120 121 122 123 124 125 126 127 128 129 130 131 132
 133 134 135 136 137 138 139 140 141 142 143 144 145 146 147 148 149 150]
```

ii) (**5 points**) Create a sequence of 125 numbers between two given numbers, 100 and 725, with step size 5 (inter-number difference).

```python
sequence1 = np.arange(100, 725, 5)
print(sequence1)
```

[4] ✓ 0.0s

```
[100 105 110 115 120 125 130 135 140 145 150 155 160 165 170 175 180 185
 190 195 200 205 210 215 220 225 230 235 240 245 250 255 260 265 270 275
 280 285 290 295 300 305 310 315 320 325 330 335 340 345 350 355 360 365
 370 375 380 385 390 395 400 405 410 415 420 425 430 435 440 445 450 455
 460 465 470 475 480 485 490 495 500 505 510 515 520 525 530 535 540 545
 550 555 560 565 570 575 580 585 590 595 600 605 610 615 620 625 630 635
 640 645 650 655 660 665 670 675 680 685 690 695 700 705 710 715 720]
```

iii) (**5 points**) Create a sequence of 125 floating point numbers between two numbers, 100 and 725.

```python
numbers2 = np.linspace(100, 725, 125)
print(numbers2)
```

[6] ✓ 0.0s

```
[100.         105.04032258 110.08064516 115.12096774 120.16129032
 125.2016129  130.24193548 135.28225806 140.32258065 145.36290323
 150.40322581 155.44354839 160.48387097 165.52419355 170.56451613
 175.60483871 180.64516129 185.68548387 190.72580645 195.76612903
 200.80645161 205.84677419 210.88709677 215.92741935 220.96774194
 226.00806452 231.0483871  236.08870968 241.12903226 246.16935484
 251.20967742 256.25       261.29032258 266.33064516 271.37096774
 276.41129032 281.4516129  286.49193548 291.53225806 296.57258065
 301.61290323 306.65322581 311.69354839 316.73387097 321.77419355
 326.81451613 331.85483871 336.89516129 341.93548387 346.97580645
 352.01612903 357.05645161 362.09677419 367.13709677 372.17741935
 377.21774194 382.25806452 387.2983871  392.33870968 397.37903226
 402.41935484 407.45967742 412.5        417.54032258 422.58064516
 427.62096774 432.66129032 437.7016129  442.74193548 447.78225806
 452.82258065 457.86290323 462.90322581 467.94354839 472.98387097
 478.02419355 483.06451613 488.10483871 493.14516129 498.18548387
 503.22580645 508.26612903 513.30645161 518.34677419 523.38709677
 528.42741935 533.46774194 538.50806452 543.5483871  548.58870968
 553.62903226 558.66935484 563.70967742 568.75       573.79032258
 578.83064516 583.87096774 588.91129032 593.9516129  598.99193548
 604.03225806 609.07258065 614.11290323 619.15322581 624.19354839
 629.23387097 634.27419355 639.31451613 644.35483871 649.39516129
 654.43548387 659.47580645 664.51612903 669.55645161 674.59677419
 679.63709677 684.67741935 689.71774194 694.75806452 699.7983871
 704.83870968 709.87903226 714.91935484 719.95967742 725.        ]
```

iv) (**5 points**) reshape the "sequence1" into a 25*5 Matrix and save it in a variable "seqMatrix".

```python
seqMatrix = sequence1.reshape(25,5)
print(seqMatrix)
```

[8] ✓ 0.0s

```
[[100 105 110 115 120]
 [125 130 135 140 145]
 [150 155 160 165 170]
 [175 180 185 190 195]
 [200 205 210 215 220]
 [225 230 235 240 245]
 [250 255 260 265 270]
 [275 280 285 290 295]
 [300 305 310 315 320]
 [325 330 335 340 345]
 [350 355 360 365 370]
 [375 380 385 390 395]
 [400 405 410 415 420]
 [425 430 435 440 445]
 [450 455 460 465 470]
 [475 480 485 490 495]
 [500 505 510 515 520]
 [525 530 535 540 545]
 [550 555 560 565 570]
 [575 580 585 590 595]
 [600 605 610 615 620]
 [625 630 635 640 645]
 [650 655 660 665 670]
 [675 680 685 690 695]
 [700 705 710 715 720]]
```

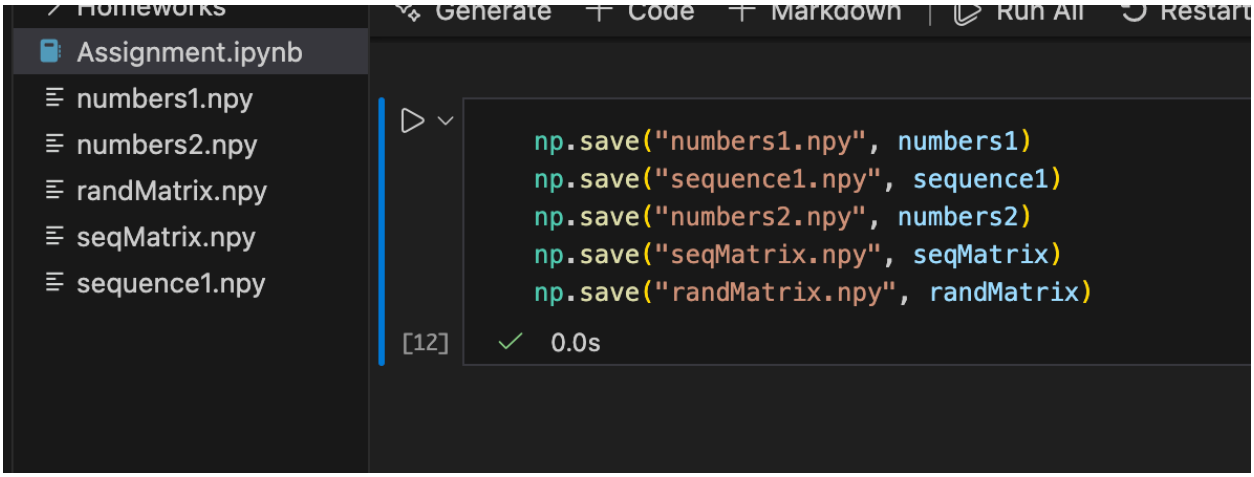
v) (**5 points**) Create a matrix of size 25*5 by creating random numbers between 10 and 300.

```python
randMatrix = np.random.randint(10, 301, size=(25,5))
print(randMatrix)
```

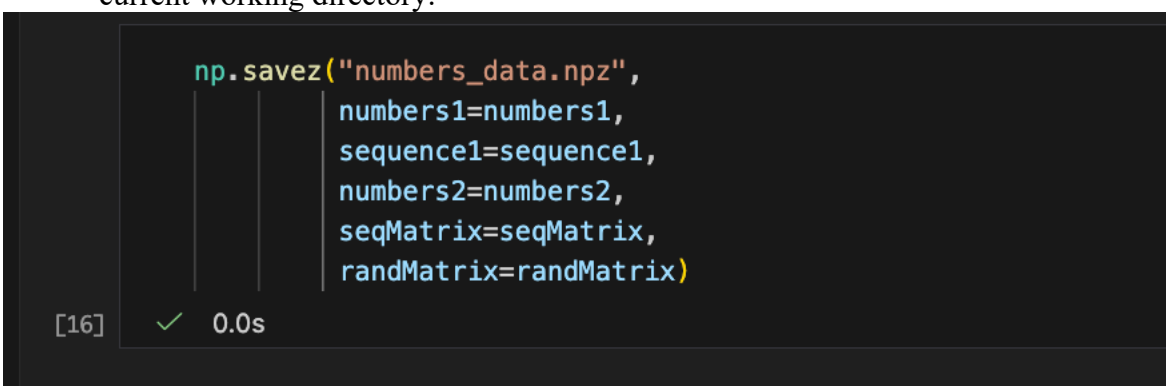
[10] ✓ 0.0s

```
[[ 53 277 105 185 233]
 [105 299 113  91 247]
 [ 36 250 174  55 300]
 [180 231 201 188 125]
 [ 26 110  46  51 241]
 [132 281 153 179 278]
 [124 114 228 180 136]
 [152 129 286  46  91]
 [164 206  96 127 178]
 [ 68 260  63  91 281]
 [299  25 160 188 128]
 [258  89  75  67 286]
 [277  85  93  28  85]
 [ 55 114 213 215 136]
 [ 94 222 155 180 299]
 [175  26  37 135  51]
 [175 186 103 117 194]
 [252 147 136 207 237]
 [190 299 287 257  42]
 [ 22 207 183 228  34]
 [109 273  22 208 240]
 [283 297 292 129 117]
 [125  22 122 242  76]
 [185 264 202 210 122]
 [133 256 166 173  60]]
```

vi) (**5 points**) Use a numPy save function to save numbers1, sequence1, numbers2, seqMatrix, and randMatrix as a numpy object (binary file like: numbers1.npy) in your current working directory.



vii) (**5 points**) Use a numPy savez function to save numbers1, sequence1, numbers2, seqMatrix, and randMatrix as a numpy object (binary file like numbers1.npz) in your current working directory.



viii) (**5 points**) Load data, *.npy and *.npz, from the disk and print them on the console.

```python
load_numbers1 = np.load("numbers1.npy")
print(load_numbers1)

data = np.load("numbers_data.npz")
print(data["numbers1"])
print(data["sequence1"])
print(data["numbers2"])
print(data["seqMatrix"])
print(data["randMatrix"])
```

[17] ✓ 0.0s

```
[ 25  26  27  28  29  30  31  32  33  34  35  36  37  38  39  40  41  42
  43  44  45  46  47  48  49  50  51  52  53  54  55  56  57  58  59  60
  61  62  63  64  65  66  67  68  69  70  71  72  73  74  75  76  77  78
  79  80  81  82  83  84  85  86  87  88  89  90  91  92  93  94  95  96
  97  98  99 100 101 102 103 104 105 106 107 108 109 110 111 112 113 114
 115 116 117 118 119 120 121 122 123 124 125 126 127 128 129 130 131 132
 133 134 135 136 137 138 139 140 141 142 143 144 145 146 147 148 149 150]
[ 25  26  27  28  29  30  31  32  33  34  35  36  37  38  39  40  41  42
  43  44  45  46  47  48  49  50  51  52  53  54  55  56  57  58  59  60
  61  62  63  64  65  66  67  68  69  70  71  72  73  74  75  76  77  78
  79  80  81  82  83  84  85  86  87  88  89  90  91  92  93  94  95  96
  97  98  99 100 101 102 103 104 105 106 107 108 109 110 111 112 113 114
 115 116 117 118 119 120 121 122 123 124 125 126 127 128 129 130 131 132
 133 134 135 136 137 138 139 140 141 142 143 144 145 146 147 148 149 150]
[100 105 110 115 120 125 130 135 140 145 150 155 160 165 170 175 180 185
 190 195 200 205 210 215 220 225 230 235 240 245 250 255 260 265 270 275
 280 285 290 295 300 305 310 315 320 325 330 335 340 345 350 355 360 365
 370 375 380 385 390 395 400 405 410 415 420 425 430 435 440 445 450 455
 460 465 470 475 480 485 490 495 500 505 510 515 520 525 530 535 540 545
 550 555 560 565 570 575 580 585 590 595 600 605 610 615 620 625 630 635
 640 645 650 655 660 665 670 675 680 685 690 695 700 705 710 715 720]
[100.         105.04032258 110.08064516 115.12096774 120.16129032
 125.2016129  130.24193548 135.28225806 140.32258065 145.36290323
 150.40322581 155.44354839 160.48387097 165.52419355 170.56451613
 175.60483871 180.64516129 185.68548387 190.72580645 195.76612903
...
 [124  32 102 135 248]
 [ 40 213  92  46 240]
 [295 285 156 197 216]
 [107 115 255  51 106]]
```

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

ix) (**5 points**) Create a data frame with three dimensions of a cuboid: length, width, and height, and their values are numbers1, sequence1, and numbers2, respectively.

```python
df = pd.DataFrame({
    "length": numbers1[:125],
    "width": sequence1,
    "height": numbers2
})

print(df.head())
```

[19] ✓ 0.0s

```
   length  width      height
0      25    100  100.000000
1      26    105  105.040323
2      27    110  110.080645
3      28    115  115.120968
4      29    120  120.161290
```

x) (**5 points**) Calculate the volume and surface area of this cuboid and save it in the data frame (df).

```python
df["Volume"] = df["length"] * df["width"] * df["height"]

df["SurfaceArea"] = 2 * (
    df["length"]*df["width"] +
    df["length"]*df["height"] +
    df["width"]*df["height"]
)

print(df.head())
```

[21] ✓ 0.0s

```
   length  width      height         Volume   SurfaceArea
0      25    100  100.000000  250000.000000  30000.000000
1      26    105  105.040323  286760.080645  32980.564516
2      27    110  110.080645  326939.516129  36102.096774
3      28    115  115.120968  370689.516129  39364.596774
4      29    120  120.161290  418161.290323  42768.064516
```

xi) (2*5 = **10 points**) Print the following values from the data frame, df.

a) Dimension of ‘df’.
b) The last three columns of ‘df’.
c) 4th row of ‘df’.
d) 3rd column of ‘df’.
e) A data point value of the 8th row and 5th column of ‘df.’

```python
# a) Dimension of df
print(df.shape)

#b) Last three columns
print(df.iloc[:, -3:])

# c) 4th row
print(df.iloc[3])

#d) 3rd column
print(df.iloc[:, 2])

#e) Value of 8th row and 5th column
print(df.iloc[7,4])
```

[23] ✓ 0.0s

```
(125, 5)
         height        Volume   SurfaceArea
0    100.000000  2.500000e+05  3.000000e+04
1    105.040323  2.867601e+05  3.298056e+04
2    110.080645  3.269395e+05  3.610210e+04
3    115.120968  3.706895e+05  3.936460e+04
4    120.161290  4.181613e+05  4.276806e+04
..          ...           ...           ...
120  704.838710  7.154113e+07  1.394177e+06
121  709.879032  7.306785e+07  1.414074e+06
122  714.919355  7.461613e+07  1.434112e+06
123  719.959677  7.618613e+07  1.454290e+06
124  725.000000  7.777800e+07  1.474610e+06

[125 rows x 3 columns]
length             28.000000
width             115.000000
height            115.120968
Volume         370689.516129
SurfaceArea     39364.596774
Name: 3, dtype: float64
0      100.000000
1      105.040323
2      110.080645
3      115.120968
...
123    719.959677
124    725.000000
Name: height, Length: 125, dtype: float64
53824.27419354839
```

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output s

xii) (2*5 = **10 points**) Linearly rescale (min-max rescaling) all dimensions of the data frame, df.

```
df_scaled = (df - df.min()) / (df.max() - df.min())
print(df_scaled.head())
```

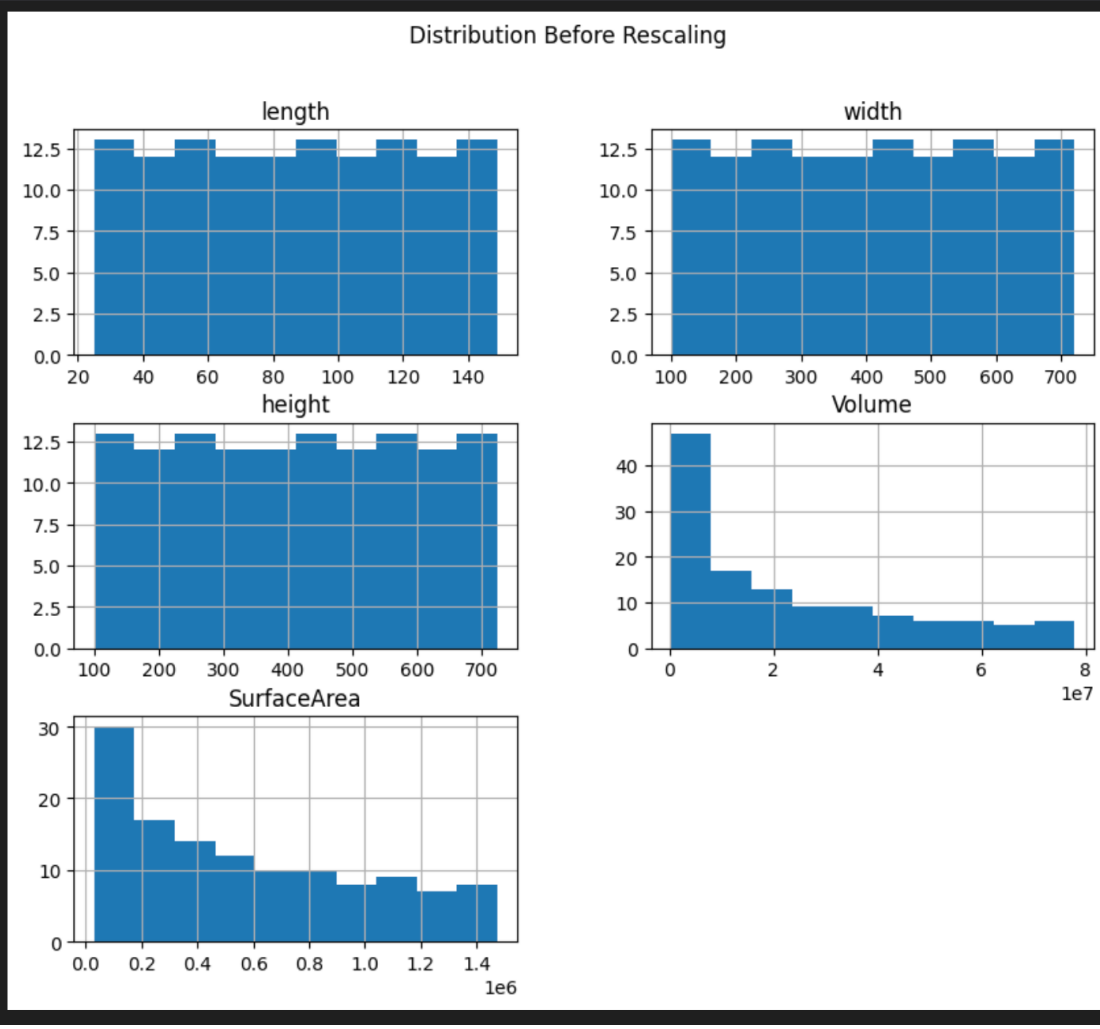
[25] ✓ 0.0s

```
...        length     width    height    Volume  SurfaceArea
0  0.000000  0.000000  0.000000  0.000000     0.000000
1  0.008065  0.008065  0.008065  0.000474     0.002063
2  0.016129  0.016129  0.016129  0.000992     0.004224
3  0.024194  0.024194  0.024194  0.001557     0.006482
4  0.032258  0.032258  0.032258  0.002169     0.008838
```

xiii) (4*5 = **20 points)** Show the data distribution of all dimensions **before and after rescaling**. Use appropriate data visualization plots.

```python
df.hist(figsize=(10,8))
plt.suptitle("Distribution Before Rescaling")
plt.show()
```

[27] ✓ 0.2s

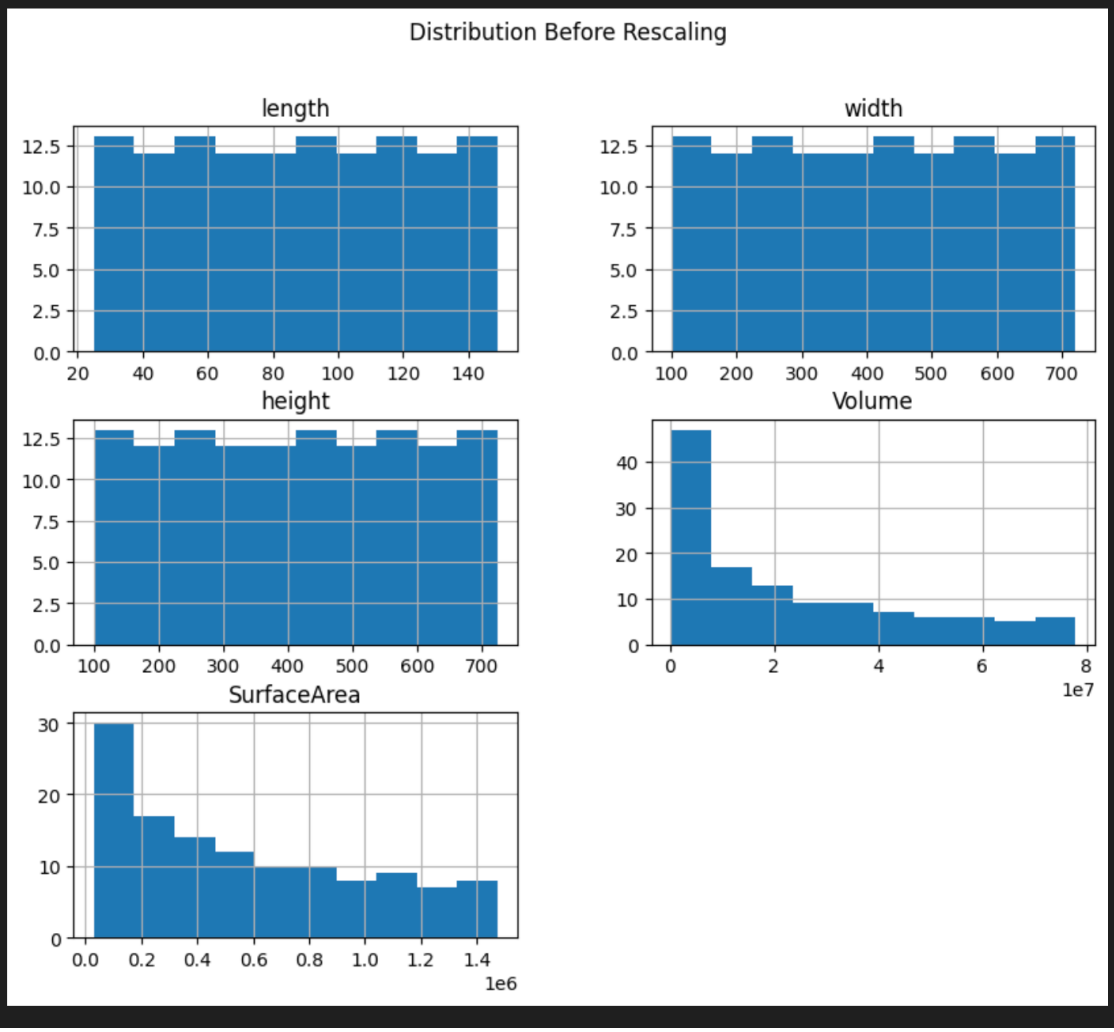


xiv) (**5 points**) Did distribution change after rescaling? If no, explain it with appropriate reasons.

```python
df.hist(figsize=(10,8))
plt.suptitle("Distribution Before Rescaling")
plt.show()
```

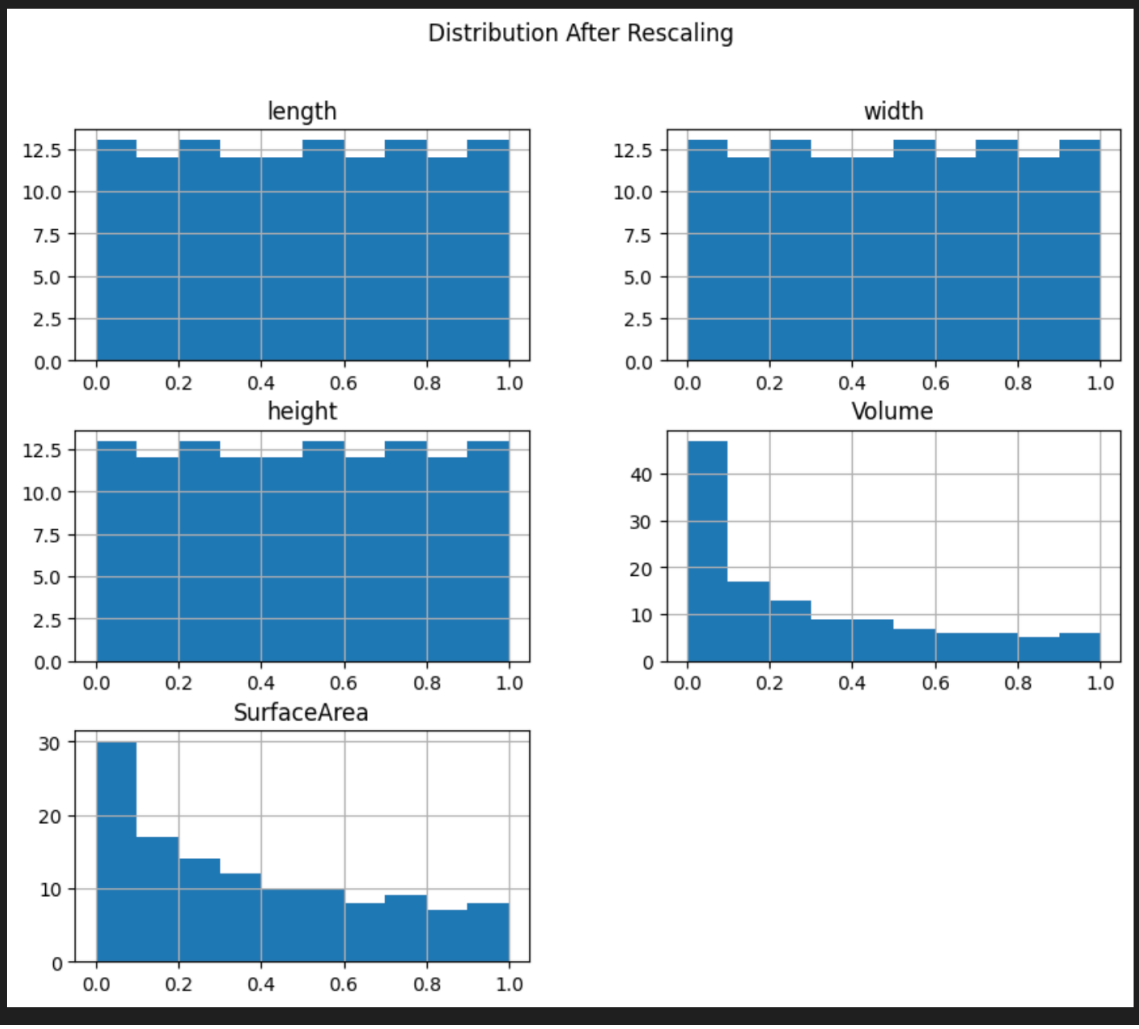
[29] ✓ 0.1s

...



```
df_scaled.hist(figsize=(10,8))
plt.suptitle("Distribution After Rescaling")
plt.show()
```

[30] ✓ 0.1s



ANS:

No, the distribution does not change after min-max rescaling. Min-max scaling only rescales the range of values (0-1) but preserves the shape of the distribution because it is a linear transformation.

xv) (5 **points**) Save 'df' data frame object in your current working directory as a binary file and load it again in '**df1**' variable. Also, show the first and last 10 rows of the **df1,** a data frame object. (Hint: use pickle package)

```python
#save
with open("df.pkl", "wb") as f:
    pickle.dump(df, f)

#load
with open("df.pkl", "rb") as f:
    df1 = pickle.load(f)

# Show first and last 10 rows
print(df1.head(10))
print(df1.tail(10))
```

[34] ✓ 0.0s

```
   length  width      height         Volume   SurfaceArea
0      25    100  100.000000  250000.000000  30000.000000
1      26    105  105.040323  286760.080645  32980.564516
2      27    110  110.080645  326939.516129  36102.096774
3      28    115  115.120968  370689.516129  39364.596774
4      29    120  120.161290  418161.290323  42768.064516
5      30    125  125.201613  469506.048387  46312.500000
6      31    130  130.241935  524875.000000  49997.903226
7      32    135  135.282258  584419.354839  53824.274194
8      33    140  140.322581  648290.322581  57791.612903
9      34    145  145.362903  716639.112903  61899.919355
     length  width      height        Volume   SurfaceArea
115     140    675  679.637097  6.422571e+07  1.296808e+06
116     141    680  684.677419  6.564687e+07  1.316000e+06
117     142    685  689.717742  6.708884e+07  1.335333e+06
118     143    690  694.758065  6.855178e+07  1.354807e+06
119     144    695  699.798387  7.003582e+07  1.374422e+06
120     145    700  704.838710  7.154113e+07  1.394177e+06
121     146    705  709.879032  7.306785e+07  1.414074e+06
122     147    710  714.919355  7.461613e+07  1.434112e+06
123     148    715  719.959677  7.618613e+07  1.454290e+06
124     149    720  725.000000  7.777800e+07  1.474610e+06
```

(Only for graduate students)

Question (2) (Total: 30 points)

i) (**10 points**) There is a CSV file, "diamonds.csv," available on the Blackboard ("Assignments" > "diamonds.csv"). Import the "diamonds.csv" file in the Python environment and save it in a Python object called "diamond." Write appropriate commands used to import the "diamonds.csv" file.

How do you create a subset of the "diamond" dataset, whose "carat" values lie between 0.2 and 0.4? Save this subset of the dataset in a variable called "dim_carat."

```python
diamond = pd.read_csv("diamonds.csv")

dim_carat = diamond[(diamond["carat"] >= 0.2) & (diamond["carat"] <= 0.4)]

print(dim_carat.head())
print(dim_carat.tail())
```

[36] ✓ 0.0s

```
   Unnamed: 0  carat      cut color clarity  depth  table  price     x     y  \
0           1   0.23    Ideal     E     SI2   61.5   55.0    326  3.95  3.98
1           2   0.21  Premium     E     SI1   59.8   61.0    326  3.89  3.84
2           3   0.23     Good     E     VS1   56.9   65.0    327  4.05  4.07
3           4   0.29  Premium     I     VS2   62.4   58.0    334  4.20  4.23
4           5   0.31     Good     J     SI2   63.3   58.0    335  4.34  4.35

      z
0  2.43
1  2.31
2  2.31
3  2.63
4  2.75
       Unnamed: 0  carat        cut color clarity  depth  table  price     x  \
53635       53636    0.3      Ideal     I    VVS1   62.1   56.0    552  4.30
53636       53637    0.3      Ideal     D     SI1   61.6   57.0    552  4.28
53637       53638    0.3  Very Good     D     SI1   62.5   57.0    552  4.26
53638       53639    0.3      Ideal     D     SI1   61.0   57.0    552  4.29
53639       53640    0.3      Ideal     D     SI1   61.0   56.0    552  4.35

          y     z
53635  4.33  2.68
53636  4.32  2.65
53637  4.28  2.67
53638  4.33  2.63
53639  4.37  2.66
```

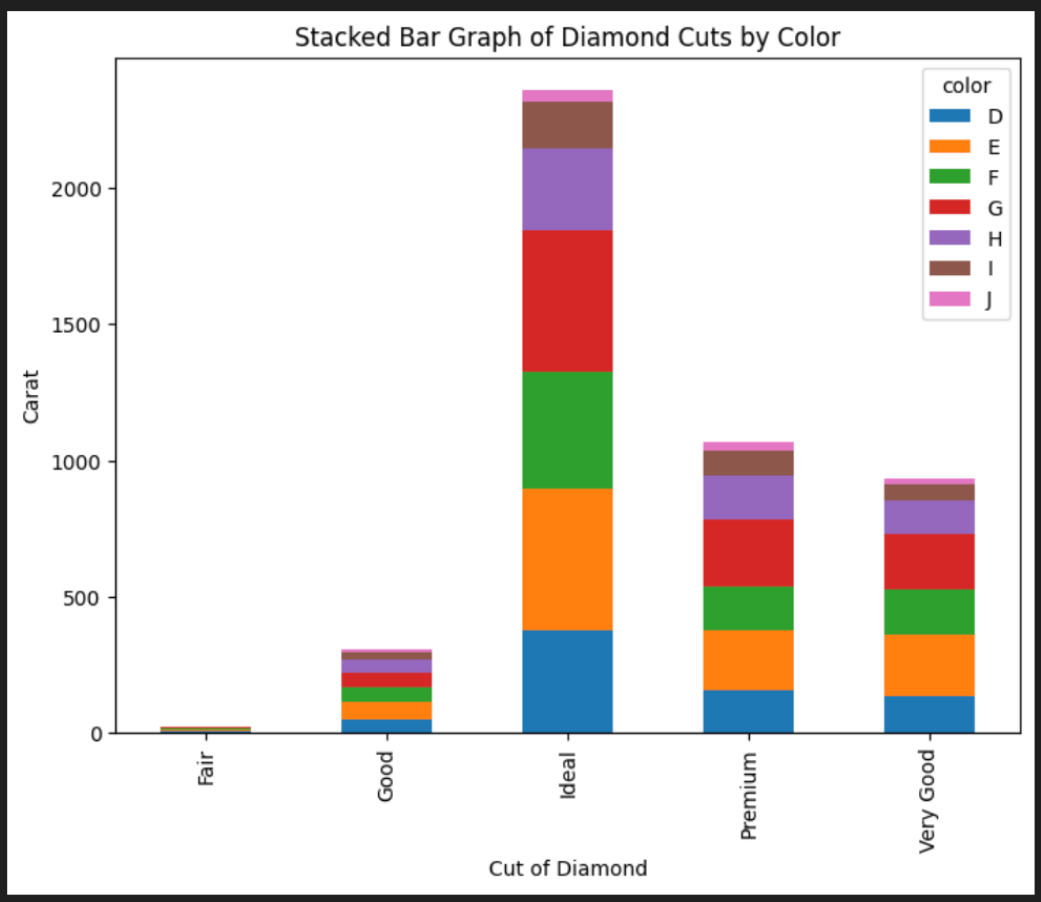
ii) **(10 points)** Plot and submit a stacked bar graph using dataset “dim_carat”. The x-axis and y-axis represent “cut” and “carat” variables. The bar's color must be filled according to the values of the “color” variable of the “dim_carat” dataset. The x-axis must be labeled as “Cut of Diamond” in this plot. You also need to submit the Python code used for this plot and the output of head(dim_carat) and tail(dim_carat).

```python
pivot_table = dim_carat.pivot_table(
    values="carat",
    index="cut",
    columns="color",
    aggfunc="sum"
)

pivot_table.plot(
    kind="bar",
    stacked=True,
    figsize=(8,6)
)

plt.xlabel("Cut of Diamond")
plt.ylabel("Carat")
plt.title("Stacked Bar Graph of Diamond Cuts by Color")
plt.show()
```

[38] ✓ 0.0s



iii) (**10 points**) Create a "diamond" dataset subset with only 25 row values and all column values. Save it in another variable called "diamond25". Plot and submit a Line Graph using the "diamond25" dataset. This plot must include the following specifications for the appearance of data points. The labels of the x and y axes need to be written as "Carat of Diamond" and "Price of Diamond," respectively. The x-axis represents the discrete value of the "carat," and the y-axis represents the "price" variable.

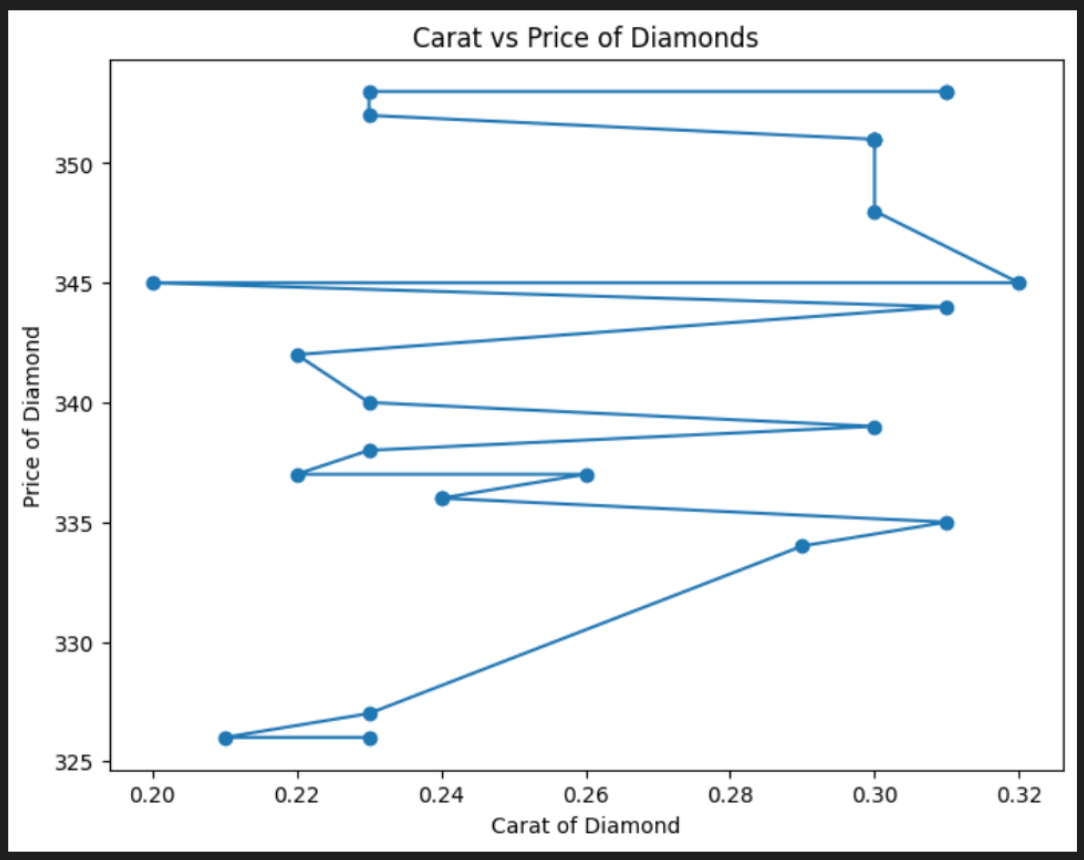
```python
diamond25 = diamond.iloc[:25]

plt.figure(figsize=(8,6))
plt.plot(diamond25["carat"], diamond25["price"], marker='o')

plt.xlabel("Carat of Diamond")
plt.ylabel("Price of Diamond")
plt.title("Carat vs Price of Diamonds")

plt.show()
```

[40] ✓ 0.0s



*****

Important! You need to do your assignment. Do not copy and make any changes to others' assignments. If you do so, you will not learn the data mining concepts by copying other answers. The examples we have discussed in lectures or book chapters would help you solve these problems. Be creative! Do something that is uniquely yours. Do something interesting and fun!

Grading Rubrics

Q1. (Total: 100 points)

i) (5 Points) For the correct Python code and output.
ii) (5 Points) For the correct Python code and output.
iii) (5 Points) For the correct Python code and output.
iv) (5 Points) For the correct Python code and output.
v) (5 Points) For the correct Python code and output.
vi) (5 Points) For the correct Python code and output.
vii) (5 Points) For the correct Python code and output.
viii) (5 Points) For the correct Python code and output.
ix) (5 Points) For the correct Python code and output.
x) (5 Points) For the correct Python code and output.
xi) (2*5 = 10 Points) For the correct Python code and output.
xii) (10 Points) For the correct Python code and output.
xiii) (20 Points) For the correct Python code and output.
xiv) (5 Points) For the correct Python code and output.
xv) (5 Points) For the correct Python code and output.

(Only for graduate students)

Q2. (Total: 30 Points)

i) **(10 Points)** For the correct Python code and output.
ii) **(10 Points)** For the correct Python code and output.
iii) **(10 Points)** For the correct Python code and output.

Policy on Lateness

- **One day late**: 2% less of your secured points.
- **Two days late**: 4% less of your secured points.
- **Three days late**: 6% less of your secured points.
- **Four days late**: 8% less of your secured points.
- **Five days late**: 10% less of your secured points.
- **Six days late**: 12% less of your secured points.
- **Seven days late**: 14% less of your secured points.
- **Eight days late**: 16% less of your secured points.
- **Nine days late**: 18% less of your secured points.
- **Ten days late**: 20% less of your secured points.
- **Eleven days late**: 40% less of your secured points.
- **Twelve days late**: 60% less of your secured points.
- **Thirteen days late**: 80% less of your secured points.
- **Fourteen days late**: 100% less of your secured points (0 points).